

CHƯƠNG 1

TỔNG QUAN VỀ TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG

Mục tiêu chương

Sau khi học xong chương này, người học có thể:

- Hiểu và trình bày được các khái niệm cơ bản về triển khai, quản trị hệ thống phần mềm.
- Hiểu được quy trình từ khi triển khai hệ thống, quản trị, vận hành và bảo đảm an toàn cho hệ thống hoạt động.
- Hiểu được vai trò của hệ điều hành Linux trong quản trị máy chủ hiện đại.
- Trình bày được cơ chế hoạt động của dịch vụ Nginx và Reverse Proxy.
- Cài đặt và cấu hình dịch vụ trên Linux.
- Cấu hình dịch vụ web server Nginx đóng vai trò reverse proxy.
- Triển khai SSL/TLS self-signed và xử lý lỗi liên quan tới systemd.
- Làm quen với tư duy IaC cơ bản trong quản lý cấu hình server.

1.1 Tổng quan về triển khai, quản trị hệ thống

1.1.1 Giới thiệu môn học, ý nghĩa và tầm quan trọng trong CNTT hiện đại

Trong lộ trình đào tạo kỹ sư công nghệ thông tin, môn học **Triển khai và quản trị hệ thống phần mềm** đóng vai trò là nhịp cầu cuối cùng, chuyển tiếp từ tư duy xây dựng sản phẩm sang tư duy vận hành hệ thống thực tế. Môn học cung cấp các kiến thức và kỹ năng cần thiết để đưa một ứng dụng từ môi trường phát triển (Development) lên môi trường vận hành thực tế (Production). Thay vì chỉ tập trung vào việc viết mã nguồn (Code), môn học này chuyển trọng tâm sang việc đóng gói, phân phối, cấu hình và duy trì sự ổn định của hệ thống đó trước hàng ngàn, hàng triệu người dùng.

Sau khi sinh viên đã nắm vững kỹ thuật phân tích thiết kế và thành thạo các ngôn ngữ lập trình để tạo ra mã nguồn, môn học này xuất hiện nhằm mục đích hoàn thiện hóa năng lực đưa những dòng mã đó ra khỏi môi trường giả lập để phục vụ người dùng thực trong một hệ sinh thái số đầy biến động.

Về bản chất, nếu các môn học trước đó tập trung vào tính đúng đắn của logic và hiệu quả của thuật toán, thì môn học này đặt trọng tâm vào tính bền bỉ, khả năng mở rộng và sự an toàn của dịch vụ. Vị trí của môn học nằm ở giai đoạn "hậu phát triển", nơi người học không còn xem phần mềm là một tập hợp các tệp tin rời rạc chạy trên máy tính cá nhân, mà là một thực thể sống cần được nuôi dưỡng trong môi trường máy chủ. Tại đây, những khái niệm như đóng gói container với Docker, điều phối hệ thống với Kubernetes, hay thiết lập hạ tầng bằng mã lệnh (IaC) được đưa vào để giải quyết bài toán xung đột môi trường – một trong những rào cản lớn nhất khi triển khai thực tế.

Vai trò then chốt của môn học còn thể hiện ở việc định hình cho sinh viên tư duy DevOps gắn liền với bảo mật, thường được gọi là DevSecOps – tích hợp yêu cầu an toàn

vào mọi giai đoạn thay vì chỉ kiểm tra bảo mật ở bước cuối cùng. Một hệ thống phần mềm dù có logic hoàn hảo nhưng nếu cấu hình máy chủ sai lệch, quản lý định danh lỏng lẻo hoặc thiếu hệ thống giám sát thì vẫn được coi là một thất bại trong vận hành. Thông qua việc tiếp cận với các quy trình tự động hóa tích hợp và triển khai liên tục (CI/CD – Continuous Integration/Continuous Deployment), sinh viên học được cách giảm thiểu sai sót do con người và cách tích hợp các lớp bảo mật ngay từ bước triển khai đầu tiên. Điều này không chỉ giúp hoàn thiện kỹ năng thực hành mà còn nâng cao nhận thức về trách nhiệm của một kỹ sư đối với sự ổn định của dịch vụ.

Ý nghĩa của môn học trong lộ trình phát triển kỹ năng chính là việc cung cấp cho người học cái nhìn toàn cảnh về vòng đời của một dự án công nghệ thông tin. Việc làm chủ kỹ năng quản trị và triển khai giúp sinh viên thoát khỏi giới hạn của một người viết mã thuần túy để trở thành những chuyên gia có khả năng bao quát hệ thống. Đây là mảnh ghép không thể thiếu để sinh viên tự tin bước vào môi trường doanh nghiệp, nơi mà khả năng xử lý sự cố, tối ưu hóa hiệu năng máy chủ và đảm bảo tính liên tục của dịch vụ thường được đánh giá cao tương đương với năng lực phát triển tính năng mới. Kết thúc môn học, sinh viên sẽ có đủ bản lĩnh để quản lý một hệ thống phần mềm hiện đại, đảm bảo nó không chỉ "chạy được" mà còn phải "chạy tốt, chạy nhanh và chạy an toàn".

1.1.2 Quy trình thực hiện DevOps

DevOps là viết tắt của Development (Dev) và Operations (Ops). DevOps là một văn hóa làm việc kết hợp giữa kỹ sư phát triển phần mềm (dev) với bộ phận operator (kỹ sư hệ thống, nhân viên bảo mật, kỹ sư mạng, kỹ sư hạ tầng,...) nhằm mục đích rút ngắn vòng đời phát triển sản phẩm (SDLC).

DevOps không phải là một công cụ (tool) hỗ trợ làm việc, mà là một phương pháp làm việc cần đến sự hỗ trợ của nhiều công cụ khác nhau, hay một quy trình khép kín gồm các giai đoạn lặp đi lặp lại, thường được minh họa bằng hình ảnh dấu vô cực (∞) để thể hiện tính liên tục không ngừng nghỉ. Ngoài ra, không có một tiêu chuẩn bắt buộc nào mà những người áp dụng DevOps phải tuân theo. Nó chỉ đơn giản là một văn hóa, một cách làm việc kết hợp giữa dev và nhân viên operations thay vì làm việc một mình hay chỉ một team độc lập nào đó.

Các bước của quy trình thực hiện bao gồm:

1. Giai đoạn Lập kế hoạch (Plan)

Đây là điểm khởi đầu của mọi chu kỳ. Trong giai đoạn này, đội ngũ phát triển và vận hành cùng nhau xác định các yêu cầu, mục tiêu của dự án và các tính năng cần phát triển. Thay vì lập kế hoạch cho cả một năm dài, DevOps ưu tiên các kế hoạch ngắn hạn (Sprints) để có thể phản hồi nhanh với thay đổi của thị trường.

Công cụ phổ biến: Jira, Trello, Slack.

2. Giai đoạn Lập trình (Code)

Tại đây, các lập trình viên bắt đầu viết mã nguồn. Điểm khác biệt trong DevOps là mã nguồn phải được quản lý tập trung và chặt chẽ. Mọi thay đổi đều phải được lưu vết để đảm bảo toàn bộ đội ngũ có thể cộng tác trên cùng một phiên bản mới nhất.

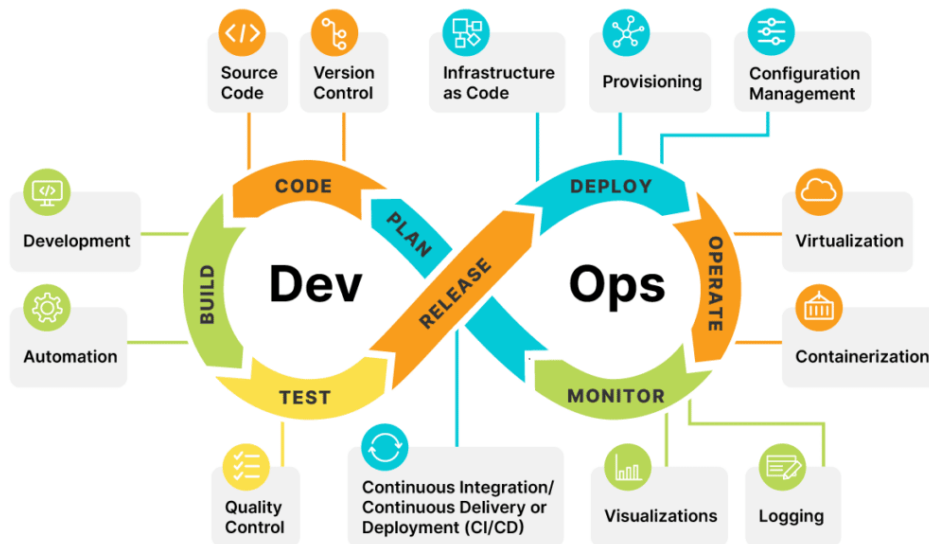
Công cụ phổ biến: Git, GitHub, GitLab.

3. Giai đoạn Xây dựng (Build)

Sau khi mã nguồn được đẩy lên hệ thống quản lý phiên bản, quy trình xây dựng sẽ

tự động được kích hoạt. Mã nguồn sẽ được biên dịch thành các tệp thực thi hoặc đóng gói vào các Container (như Docker). Đây là bước kiểm tra đầu tiên để đảm bảo mã không có lỗi cú pháp và có thể chạy được.

Công cụ phổ biến: Maven, Gradle, Docker.



Hình 1.1: Quy trình Devops chia theo các tính năng

4. Giai đoạn Kiểm thử (Test)

Đây là giai đoạn then chốt để đảm bảo chất lượng. Trong DevOps, chúng ta thực hiện Continuous Testing (Kiểm thử liên tục). Các kịch bản kiểm thử tự động (Unit Test, Integration Test) sẽ chạy ngay khi mã được xây dựng xong. Nếu có bất kỳ lỗi nào, hệ thống sẽ báo ngay cho lập trình viên để sửa chữa.

Công cụ phổ biến: JUnit, Selenium, Jenkins.

5. Giai đoạn Phát hành & Triển khai (Release & Deploy)

Khi mã đã vượt qua các bài kiểm thử, nó sẽ sẵn sàng để phát hành. Quy trình này được gọi là Continuous Deployment (Triển khai liên tục). Phần mềm sẽ được đẩy tự động lên các môi trường thử nghiệm (Staging) hoặc môi trường thực tế (Production) mà không cần sự can thiệp thủ công, giúp giảm thiểu rủi ro do con người gây ra.

Công cụ phổ biến: Jenkins, GitLab CI/CD, Kubernetes.

6. Giai đoạn Vận hành (Operate)

Sau khi triển khai, đội ngũ vận hành sẽ quản lý hạ tầng để đảm bảo ứng dụng chạy ổn định. Ở đây, khái niệm Infrastructure as Code (IaC) được áp dụng, cho phép chúng ta thiết lập máy chủ, mạng và lưu trữ thông qua mã lệnh, giúp việc tái thiết lập môi trường trở nên cực kỳ nhanh chóng.

Công cụ phổ biến: Terraform, Ansible.

7. Giai đoạn Giám sát (Monitor)

Dòng phản hồi (Feedback loop) hoàn chỉnh nhờ vào việc giám sát. Chúng ta cần theo dõi hiệu năng hệ thống, trải nghiệm người dùng và các lỗi hỏng bảo mật phát sinh. Các dữ liệu thu thập được từ giai đoạn này sẽ là đầu vào quý giá cho giai đoạn Lập kế hoạch (Plan) của chu kỳ tiếp theo.

Công cụ phổ biến: Prometheus, Grafana, ELK Stack.

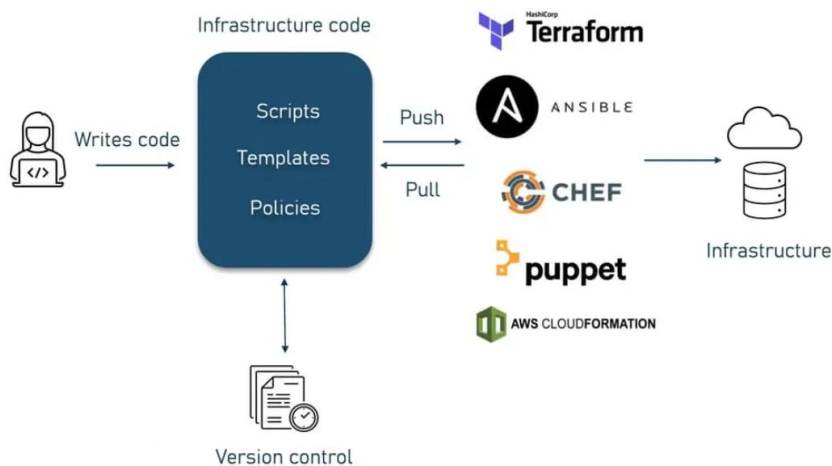
Một lưu ý đó là: ranh giới giữa các bước này trong thực tế rất mong manh. Chúng hòa quyện vào nhau tạo thành một luồng công việc tự động hóa hoàn toàn. Mục tiêu cuối cùng của quy trình này không chỉ là ”chạy được ứng dụng”, mà là tạo ra một bộ máy sản xuất phần mềm nhanh hơn, an toàn hơn và tin cậy hơn.

1.1.3 Tư duy IaC trong quản lý cấu hình máy chủ hiện đại

Nếu trước đây, việc thiết lập một máy chủ được coi là một công việc ”thủ công mỹ nghệ” đầy rủi ro, công việc thường mặc định dành cho các quản trị viên hệ thống. Thì hiện nay, với IaC, chúng ta đang bước vào kỷ nguyên của sự chính xác và tự động hóa tuyệt đối, đó là sự chuyển dịch từ ”**Thủ công**” sang ”**Khai báo**”.

Để vận hành một hệ thống phần mềm, chúng ta cần máy chủ, hệ điều hành, thư viện, cơ sở dữ liệu và các thiết lập mạng.

1. *Cách làm truyền thống* (Manual Configuration): Người quản trị đăng nhập vào từng máy chủ, gõ từng dòng lệnh, cài đặt từng gói phần mềm. Hãy tưởng tượng nếu hệ thống của công ty cần 100 máy chủ giống hệt nhau, việc lặp lại thao tác này 100 lần không chỉ gây mệt mỏi mà còn chắc chắn dẫn đến sai sót. Chỉ cần một máy chủ thiếu một cấu hình nhỏ, toàn bộ hệ thống có thể sụp đổ.
2. *Tư duy IaC*: Thay vì thao tác trực tiếp, quản trị viên sẽ viết một tệp tin văn bản (thường là YAML hoặc JSON) mô tả trạng thái mong muốn của hệ thống. Ví dụ: ”Tôi muốn một máy chủ Ubuntu, có cài đặt Nginx phiên bản 1.24 và mở cổng 80”. Sau đó, chúng ta cần dùng công cụ để ”đọc” tệp tin này và nó sẽ tự động tạo ra máy chủ đúng như mô tả.



Hình 1.2: Tư duy về IaC trong phát triển phần mềm hiện đại

Tại sao người học cần tư duy IaC khi muốn vận hành độc lập? Khi chúng ta muốn tự mình triển khai một dự án cá nhân hoặc khởi nghiệp, nguồn lực về thời gian và con người là rất hạn chế. Khi đó, IaC mang lại cho chúng ta ba giá trị cốt lõi:

- Tính nhất quán (Consistency): Người dùng có thể tạo ra môi trường Thử nghiệm (Staging) giống hệt môi trường Thật (Production). Điều này loại bỏ hoàn toàn câu nói kinh điển của lập trình viên: ”Nhưng máy em vẫn chạy bình thường mà!”.

- Khả năng tái bản (Repeatability): Nếu máy chủ gặp sự cố hoặc bị tấn công, thay vì mất hàng giờ để cấu hình lại từ đầu, chúng ta chỉ cần chạy lại mã nguồn IaC. Trong phút chốc, một "bản sao" hoàn hảo của hệ thống cũ sẽ được dựng lên.
- Quản lý phiên bản (Version Control): Vì hạ tầng giờ đây là các tệp mã nguồn, chúng ta có thể đưa toàn bộ lên Git. Mọi sự thay đổi cấu hình đều được ghi lại, vào lúc nào và có thể "quay xe" (Rollback) về phiên bản trước đó nếu cấu hình mới gây lỗi.

Nguyên lý "Bất biến" (Immutable Infrastructure) của hệ thống: Trong quản trị hiện đại, chúng ta không "sửa" máy chủ khi nó lỗi. Nếu một máy chủ bị cấu hình sai, tư duy IaC khuyên chúng ta: Xóa nó đi và tạo một cái mới từ mã nguồn. Điều này giúp tránh hiện tượng "Configuration Drift" – tức là theo thời gian, các máy chủ vốn giống nhau dần trở nên khác biệt do các lần sửa chữa nhỏ lẻ, dẫn đến những lỗi hệ thống cực kỳ khó tìm ra nguyên nhân.

Định hướng học tập trong tương lai Để làm chủ kỹ năng này, người học nên bắt đầu tiếp cận theo lộ trình sau:

- Hãy học ngôn ngữ đánh dấu: Nắm vững cấu trúc tệp YAML vì đây là ngôn ngữ chung của hầu hết các công cụ IaC.
- Làm quen với ảo hóa và Container: Hiểu cách Docker đóng gói ứng dụng.
- Công cụ quản lý cấu hình (Configuration Management): Tìm hiểu Ansible. Đây là công cụ rất dễ tiếp cận cho sinh viên vì nó không cần cài đặt phần mềm trung gian trên máy chủ đích.
- Công cụ khởi tạo hạ tầng (Provisioning): Tìm hiểu Terraform. Đây là tiêu chuẩn công nghiệp giúp chúng ta định nghĩa toàn bộ mạng lưới, máy chủ trên các nền tảng đám mây như AWS, Azure hay Google Cloud.

Vận hành một hệ thống phần mềm độc lập không có nghĩa là chúng ta sẽ phải làm mọi thứ bằng tay. Người vận hành giỏi là người "lười biếng" một cách thông minh: họ dành thời gian viết mã để máy móc tự chăm sóc lẫn nhau. Tư duy IaC không chỉ là kỹ thuật, nó là sự tự do – giúp người quản trị tập trung vào việc sáng tạo tính năng phần mềm mà không phải lo sợ về việc máy chủ "biến mất" giữa đêm.

1.2 Quản trị hệ điều hành Linux

1.2.1 Vai trò của Linux trong triển khai và vận hành hệ thống

Trong thế giới công nghệ, nếu mã nguồn phần mềm được coi là linh hồn, thì hệ điều hành chính là cơ thể. Và đối với các hệ thống máy chủ, đám mây (Cloud) hay DevOps hiện đại, hệ điều hành Linux chính là cơ thể mạnh mẽ và phổ biến nhất. Tại sao không phải là Windows hay macOS? Hãy cùng phân tích sâu dưới góc độ kỹ thuật và vận hành hệ thống.

a. Triết lý Linux: Sự tự do và khả năng tùy biến tối đa

Như chúng ta đều biết, điểm khác biệt lớn nhất giữa Linux và các hệ điều hành thương mại là triết lý "Mọi thứ đều là tệp tin" (Everything is a file). Từ tiến trình (process), thiết bị phần cứng (hardware) cho đến các kết nối mạng (network socket), tất cả đều được biểu diễn dưới dạng tệp tin.

Điều này cho phép các lập trình viên có thể thực hiện các thao tác can thiệp vào tầng sâu nhất của hệ thống chỉ thông qua các câu lệnh thao tác văn bản. Chúng ta có thể

kernel (nhân hệ điều hành) để tối ưu hóa riêng cho một ứng dụng cụ thể, điều mà các hệ điều hành đóng như MS Windows không bao giờ cho phép.

Bên cạnh đó, Linux còn cho thấy sự tinh gọn khi triển khai hệ thống tới người dùng: Linux có phiên bản "Minimal" – tức là không giao diện đồ họa (GUI). Một máy chủ Linux có thể vận hành chỉ với vài trăm MB RAM, dành toàn bộ tài nguyên còn lại cho ứng dụng. Đây là yếu tố sống còn khi triển khai hệ thống lớn nhằm tối ưu hóa chi phí phần cứng.

b. Linux là "Ngôn ngữ chung" của Cloud và DevOps

Thực tế hiện nay hơn 90% hạ tầng đám mây công cộng (AWS, Azure, Google Cloud) chạy trên nền tảng Linux. Tất cả các công cụ đã đề cập ở trên như Docker, Kubernetes hay Ansible đều được sinh ra và tối ưu hóa trước hết cho môi trường Linux.

Containerization (Công nghệ đóng gói): Docker thực chất dựa trên các tính năng lõi của Linux Kernel là Namespaces (cách ly tài nguyên) và Cgroups (giới hạn tài nguyên). Nếu không có Linux, sẽ không có cuộc cách mạng Container.

Môi trường đồng nhất: Khi người dùng sử dụng Linux từ máy tính cá nhân (Development) cho đến máy chủ thật (Production), họ sẽ loại bỏ được rủi ro sai lệch môi trường. Một lệnh `sh` chạy trên Ubuntu cá nhân chắc chắn sẽ chạy đúng trên một cụm máy chủ Debian tại trung tâm dữ liệu.

c. Giao diện dòng lệnh (CLI) và tư duy tự động hóa

Làm việc với Linux là làm việc với Terminal. Đây có vẻ là một rào cản với sinh viên mới, nhưng nó lại là "vũ khí" mạnh nhất của người làm vận hành. Linux cung cấp hàng nghìn câu lệnh với rất nhiều các tham số tùy biến cho người dùng.

Khả năng lập trình (Scripting): Mọi thao tác trên Linux đều có thể được viết thành kịch bản (Shell Script). Thay vì nhấn chuột 10 bước trên giao diện đồ họa, người dùng chỉ cần thực hiện một dòng lệnh. Dòng lệnh này có thể được nhân bản ra hàng ngàn máy chủ trong tích tắc.

Hỗ trợ quản trị từ xa mạnh mẽ: Thông qua giao thức SSH (Secure Shell), quản trị viên có thể quản trị một máy chủ cách nửa vòng trái đất với băng thông cực thấp mà vẫn đảm bảo tính bảo mật và toàn vẹn.

d. Hệ sinh thái mã nguồn mở và tính an toàn bảo mật

Dưới góc độ An toàn thông tin, Linux sở hữu một lợi thế vô song nhờ cộng đồng mã nguồn mở.

Sự minh bạch trong cách quản lý và vận hành: Bất kỳ ai cũng có thể kiểm tra mã nguồn của Linux để tìm lỗ hổng. Khi một lỗi bảo mật (CVE) xuất hiện, cộng đồng toàn cầu thường đưa ra bản vá chỉ trong vài giờ, nhanh hơn rất nhiều so với quy trình của các tập đoàn phần mềm đóng.

Tính năng phân quyền chặt chẽ: Cơ chế phân quyền người dùng (Root vs User) và các lớp bảo mật bổ sung như SELinux hay AppArmor giúp cô lập ứng dụng. Nếu một ứng dụng web bị tấn công, hacker cũng rất khó để thoát ra khỏi các vòng bảo vệ đó để phá hoại toàn bộ máy chủ.

e. Định hướng tư duy thực hành cho sinh viên

Để chuẩn bị cho việc vận hành một hệ thống độc lập trong tương lai, chúng ta cần làm quen với việc không chỉ học "câu lệnh", mà phải học "cấu trúc" câu lệnh và thực hành thường xuyên để ghi nhớ, đồng thời:

1. Cần học cách quản lý gói (Package Management): Hiểu sự khác biệt giữa apt (Debian/Ubuntu) và yum/dnf (RHEL/CentOS). Đây là cách chúng ta cài đặt thư viện cho phần mềm.
2. Làm chủ cấu trúc thư mục: phải biết `/etc` chứa cấu hình, `/var/log` chứa nhật ký vận hành, và `/opt` là nơi cài đặt các ứng dụng bên thứ ba.
3. Tư duy đọc nhật ký (log): trong vận hành, khi phần mềm lỗi, Linux sẽ cho quản trị viên biết nguyên nhân thông qua log. Kỹ năng đọc và dùng `grep` để tìm kiếm trong log là kỹ năng phân biệt giữa một người mới và một chuyên gia.

Như vậy, Linux không chỉ là một hệ điều hành; nó là một hệ sinh thái kiến thức. Làm chủ Linux nghĩa là chúng ta đã nắm giữ chìa khóa để mở cánh cửa vào thế giới **Cloud, DevOps và Cyber Security**. Hãy bắt đầu bằng việc cài đặt một bản phân phối Linux (như Ubuntu Server) trên máy ảo và thực hiện triển khai website đầu tiên của mình hoàn toàn bằng dòng lệnh. Đó chính là viên gạch đầu tiên cho sự nghiệp vận hành hệ thống chuyên nghiệp của các quản trị viên hệ thống trong tương lai.

1.2.2 Cấu hình dịch vụ cơ bản trên Linux

Ngoài việc hiểu vai trò của Linux, người quản trị còn cần biết cách áp dụng vai trò đó vào việc cấu hình dịch vụ cơ bản trên máy chủ. Quy trình chung khi cấu hình bất kỳ dịch vụ nào trên Linux gồm 4 bước:

1. Cài đặt (Install): Sử dụng trình quản lý gói (**apt** hoặc **dnf**) để tải về.
2. Cấu hình (Configure): Tìm đến thư mục `/etc`, chỉnh sửa tệp cấu hình theo yêu cầu.
3. Kiểm soát trạng thái (Control): Sử dụng **systemctl** để khởi động (start), dừng (stop) hoặc tải lại (reload) cấu hình.
4. Kiểm tra & Bảo mật (Check & Secure): Kiểm tra log tại `/var/log` và mở cổng tương ứng trên tường lửa.

Hai ví dụ áp dụng quy trình này là dịch vụ quản trị từ xa SSH và dịch vụ truyền tệp FTP, được minh họa dưới đây.

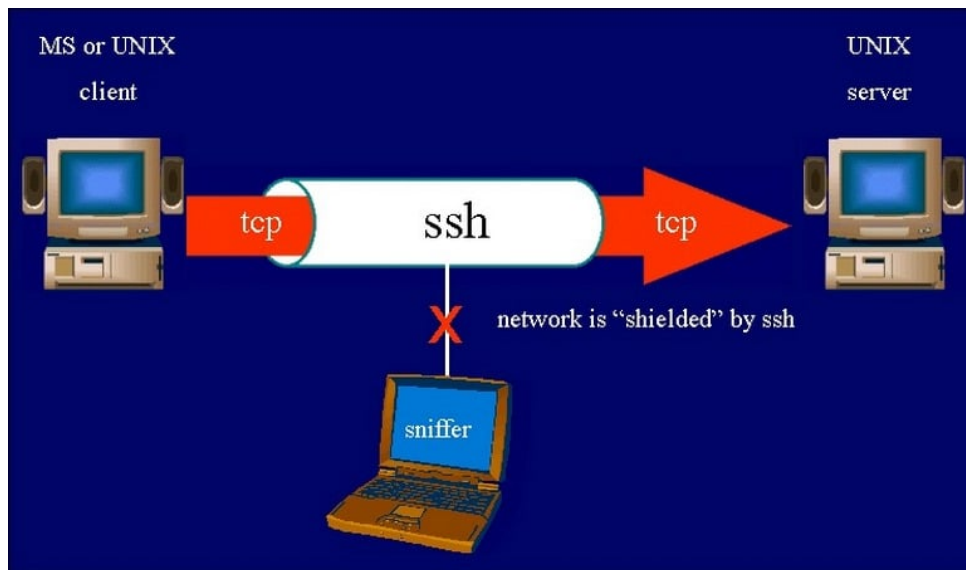
Cài đặt và cấu hình dịch vụ quản trị từ xa (SSH – Secure Shell)

SSH là dịch vụ đầu tiên mà bất kỳ quản trị viên hệ thống nào đều cần phải biết vì nó cho phép quản lý máy chủ từ xa mà không cần ngồi trực tiếp trước màn hình máy vật lý. SSH mã hóa toàn bộ dữ liệu truyền đi, giúp bảo vệ mật khẩu và lệnh điều khiển khỏi những kẻ nghe lén trên mạng.

Về nguyên lý hoạt động, SSH thiết lập một kênh giao tiếp được mã hóa giữa hai máy qua môi trường Internet, xác thực bằng mật khẩu hoặc bằng cặp khóa công khai/riêng tư – cách thứ hai được khuyến khích vì an toàn hơn xác thực bằng mật khẩu. Nhờ cơ chế này, SSH thường được triển khai trên các máy chủ ảo (VPS) và máy chủ vật lý hiện đại để bảo đảm an toàn và khả năng mở rộng khi quản trị từ xa.

Hầu hết các bản phân phối Linux như Ubuntu đã cài sẵn SSH; nếu chưa có, có thể cài đặt bằng các lệnh:

```
$sudo apt-get update
$sudo apt-get install openssh-server
```



Hình 1.3: Cơ chế hoạt động của dịch vụ SSH

Sau khi cài đặt thành công, tệp cấu hình của SSH nằm tại `/etc/ssh/sshd_config`. Để tăng tính bảo mật, chúng ta nên đổi cổng mặc định (22) sang một cổng khác.

Để tiến hành cấu hình, cần chỉnh sửa tệp cấu hình bằng lệnh `sudo nano /etc/ssh/sshd_config`. Tệp cấu hình có rất nhiều tùy chọn khác mà quản trị viên có thể thiết lập cho máy chủ SSH. Sau khi cấu hình xong, lưu thiết lập và khởi động lại dịch vụ bằng lệnh `sudo systemctl restart ssh`.

Cài đặt và cấu hình dịch vụ FTP (với vsftpd)

Dịch vụ FTP cho phép truyền tải dữ liệu dạng tệp từ máy khách lên máy chủ, đây là dịch vụ rất phổ biến trong việc quản lý tài nguyên các trang web sau khi đưa lên môi trường production. VSFTPD (Very Secure FTP Daemon) là một trong những dịch vụ FTP nhanh và an toàn nhất trên Linux. Để sử dụng dịch vụ FTP này trên máy chủ Linux, có thể thực hiện các bước cài đặt và cấu hình sau:

Bước 1: Cài đặt dịch vụ

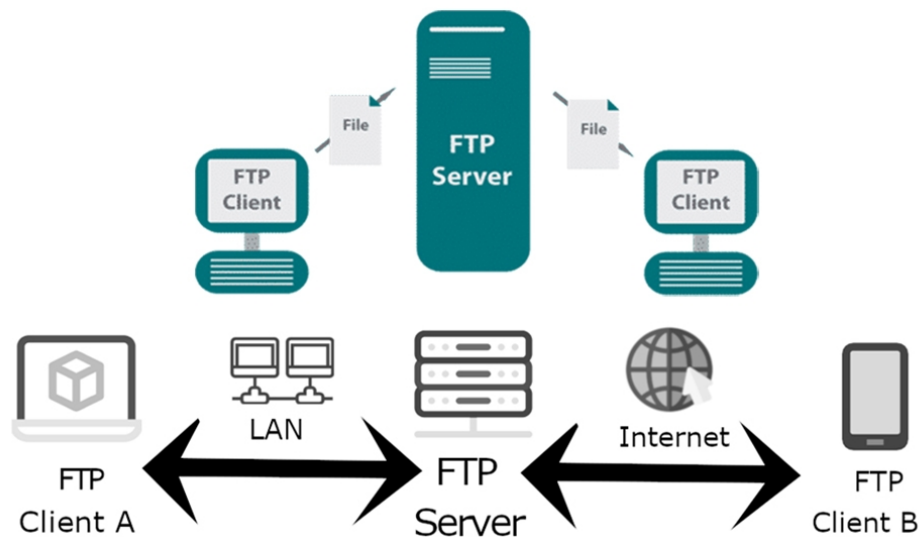
```
$sudo apt-get update
$sudo apt-get install vsftpd
```

Bước 2: Cấu hình cơ bản. Tệp cấu hình chính nằm tại `/etc/vsftpd.conf`. Chúng ta cần chỉnh sửa để cho phép người dùng có quyền ghi (upload) tệp tin lên máy chủ bằng lệnh `sudo nano /etc/vsftpd.conf`, sau đó tìm và bỏ dấu `#` trước các dòng sau để kích hoạt:

- `write_enable=YES` (cho phép tải tệp lên)
- `local_enable=YES` (cho phép người dùng cục bộ đăng nhập)
- `chroot_local_user=YES` (giới hạn người dùng trong thư mục cá nhân của họ để bảo mật)

Bước 3: Khởi động lại và kiểm tra

```
$sudo systemctl restart vsftpd
```



Hình 1.4: Dịch vụ FTP và cơ chế hoạt động

```
$sudo systemctl enable vsftpd
```

Bước 4: Để dịch vụ FTP hoạt động được, cần cấu hình tường lửa của hệ thống để bỏ chặn cổng dịch vụ. Mở tường lửa (UFW) cho FTP sử dụng cổng 21 để điều khiển và dải cổng khác cho truyền dữ liệu:

```
$sudo ufw allow 20/tcp
$sudo ufw allow 21/tcp
$sudo ufw status
```

Sau khi cấu hình xong, có thể tạo tài khoản với quyền tương ứng để sử dụng dịch vụ FTP trong việc upload tài liệu từ máy trạm lên máy chủ (tự nghiên cứu và cấu hình).

FTP truyền thông gửi mật khẩu dưới dạng văn bản thuần túy (clear text), rất dễ bị đánh cắp. Trong hệ thống thực tế, nên chuyển sang sử dụng SFTP hoặc FTPS để mã hóa dữ liệu.

1.2.3 Các lệnh cơ bản trong quản lý và cài đặt các gói trên Linux

Trong quá trình cài đặt và quản lý các gói phần mềm trên Linux, người quản trị hệ thống sẽ thường xuyên sử dụng các câu lệnh sau đây:

- **apt update:** cập nhật danh sách các gói phần mềm từ các nguồn lưu trữ (repository). Lệnh này không nâng cấp phần mềm mà chỉ giúp máy chủ biết được có phiên bản mới nào hiện hành hay không. Cú pháp: `sudo apt update`.
- **apt upgrade:** nâng cấp tất cả phần mềm đang có trên hệ thống lên phiên bản mới nhất dựa trên danh sách đã cập nhật từ `apt update`. Cú pháp: `sudo apt upgrade`.
- **apt install:** tải về và cài đặt một gói phần mềm mới cùng tất cả thư viện phụ thuộc (dependencies) của nó. Cú pháp: `sudo apt install <tên_gói>` (ví dụ: `git`).
- **apt remove:** gỡ bỏ một phần mềm đã cài đặt nhưng vẫn giữ lại các tệp tin cấu hình (config files), hữu ích nếu muốn cài lại sau này mà không phải thiết lập lại

từ đầu. Cú pháp: `sudo apt remove <tên_gói>`.

- **apt purge**: gỡ bỏ hoàn toàn một phần mềm, bao gồm cả các tệp tin cấu hình – cách ”dọn dẹp sạch sẽ” nhất khi một dịch vụ bị lỗi cấu hình nghiêm trọng. Cú pháp: `sudo apt purge <tên_gói>`.
- **apt autoremove**: tự động quét và gỡ bỏ các gói thư viện phụ thuộc còn sót lại sau khi phần mềm chính đã bị xóa, giúp tiết kiệm dung lượng đĩa. Cú pháp: `sudo apt autoremove`.
- **apt search**: tìm kiếm một phần mềm trong kho lưu trữ khi không nhớ chính xác tên gói. Cú pháp: `apt search <từ_khóa>`, ví dụ `apt search database`.
- **apt show**: hiển thị thông tin chi tiết về một gói phần mềm như phiên bản, dung lượng, nhà phát triển và các yêu cầu hệ thống. Cú pháp: `apt show <tên_gói>`.
- **apt list --installed**: liệt kê tất cả phần mềm hiện đang có mặt trên hệ thống. Cú pháp: `sudo apt list --installed`.
- **apt-add-repository**: thêm một nguồn lưu trữ phần mềm bên thứ ba (PPA), dùng khi cần cài đặt phần mềm đặc thù không có sẵn trong kho mặc định của hệ điều hành. Cú pháp: `sudo add-apt-repository <tên_nguồn>`.

Cần lưu ý tiền tố `sudo`: Linux là hệ điều hành có tính phân quyền rất cao, mọi thay đổi hệ thống (cài đặt/xóa bỏ) đều yêu cầu quyền quản trị cao nhất. Hãy luôn kiểm tra kỹ tên gói phần mềm trước khi nhấn Enter để tránh gỡ nhầm các thành phần quan trọng của hệ thống.

1.3 Triển khai Nginx và bảo mật dịch vụ với SSL

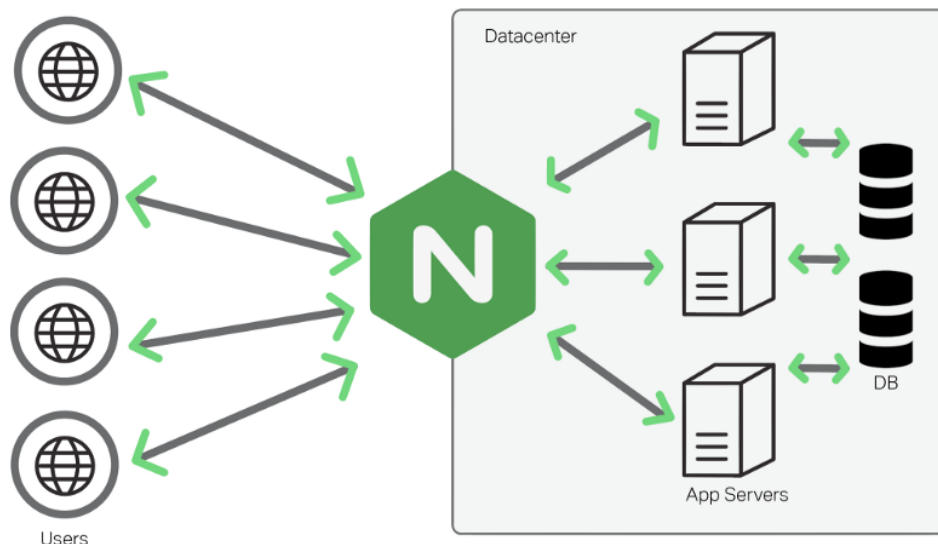
1.3.1 Giới thiệu về Nginx

Nginx (phát âm là ”Engine-X”) là một trong những máy chủ web (web server) và máy chủ proxy ngược (reverse proxy) mã nguồn mở phổ biến nhất thế giới hiện nay. Nginx sử dụng kiến trúc đơn luồng, hướng sự kiện nên hiệu quả hơn máy chủ Apache trong nhiều kịch bản tải cao. Ngoài vai trò máy chủ web, Nginx còn đảm nhiệm tốt các vai trò như cân bằng tải (load balancing) và bộ nhớ đệm HTTP (HTTP caching). Nginx là kiến thức không thể thiếu đối với một lập trình viên web, quản trị viên hệ thống hay kỹ sư DevOps.

Năm 2002, Igor Sysoev phát triển Nginx ban đầu và công bố lần đầu vào năm 2004. Tháng 7 năm 2011, công ty Nginx Inc được thành lập với trụ sở đầu tiên tại San Francisco, California, USA.

Những tính năng chính của máy chủ HTTP Nginx bao gồm:

- Có khả năng xử lý hơn 10.000 kết nối cùng lúc với bộ nhớ thấp.
- Phục vụ tệp tin tĩnh (static files) và lập chỉ mục tệp tin.
- Tăng tốc reverse proxy bằng bộ nhớ đệm (cache), cân bằng tải đơn giản và khả năng chịu lỗi.
- Hỗ trợ tăng tốc với bộ nhớ đệm của FastCGI, uwsgi, SCGI và các máy chủ mem-cached.
- Kiến trúc modular, tăng tốc độ nạp trang bằng nén gzip tự động.
- Hỗ trợ mã hoá SSL và TLS.
- Cấu hình linh hoạt; lưu lại nhật ký truy vấn.



Hình 1.5: Hệ thống Nginx trong thực tiễn

- Chuyển hướng các mã lỗi 3xx–5xx.
- Rewrite URL (URL rewriting) dùng regular expressions.
- Hạn chế tỷ lệ đáp ứng truy vấn, giới hạn số kết nối đồng thời hoặc truy vấn từ một địa chỉ.
- Khả năng nhúng mã Perl.
- Hỗ trợ và tương thích với IPv6, WebSockets, và truyền tải tập tin FLV, MP4.

Nginx được thiết kế để giải quyết bài toán C10k (hỗ trợ 10.000 kết nối đồng thời), nhờ đó sở hữu những đặc tính kỹ thuật vượt trội so với các máy chủ web khác:

- **Kiến trúc hướng sự kiện (event-driven):** khác với Apache tạo ra một tiến trình mới cho mỗi yêu cầu, Nginx sử dụng một tiến trình duy nhất để xử lý hàng ngàn kết nối cùng lúc, giúp giảm đáng kể mức tiêu tốn RAM và CPU.
- **Hiệu suất xử lý tệp tĩnh cực cao:** Nginx xử lý các nội dung như hình ảnh, CSS, JavaScript hay tệp HTML rất nhanh, giúp giảm tải cho ứng dụng phía sau.
- **Khả năng chịu tải và mở rộng:** Nginx hoạt động ổn định dưới áp lực truy cập lớn mà không chiếm dụng nhiều tài nguyên hệ thống.
- **Cấu hình linh hoạt và dễ hiểu:** tệp cấu hình của Nginx dùng cú pháp phân cấp, giúp người vận hành dễ dàng thiết lập các quy tắc phức tạp chỉ với vài dòng lệnh.
- **Tính đa năng:** Nginx không chỉ là máy chủ web, mà còn đảm nhiệm tốt vai trò của một bộ cân bằng tải và lớp bảo mật cho hệ thống.

Trong thực tế triển khai và quản trị hệ thống, Nginx thường được sử dụng vào 4 mục đích chính:

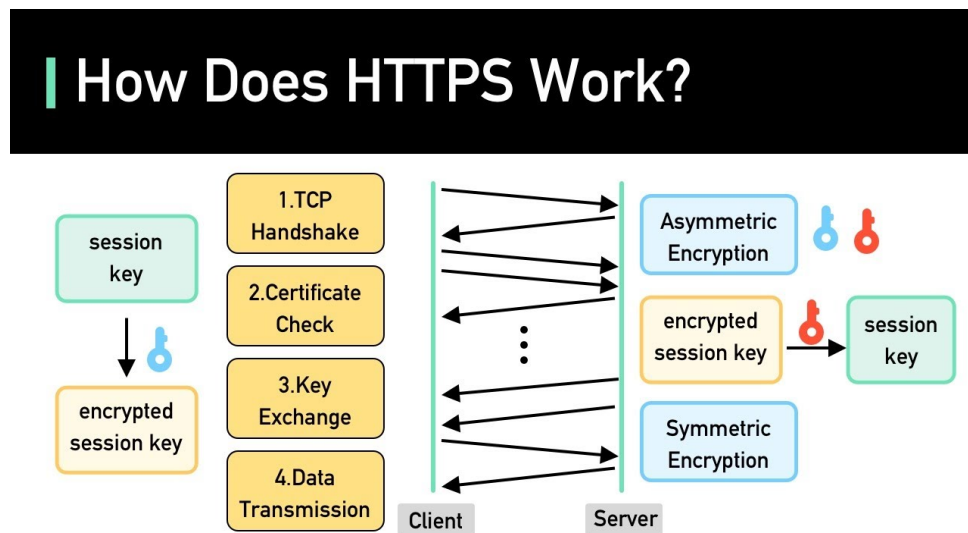
- *Làm máy chủ proxy ngược (reverse proxy):* đây là ứng dụng phổ biến nhất. Nginx đứng trước các ứng dụng (như Node.js, Python, Java) để tiếp nhận yêu cầu từ người dùng và chuyển tiếp tới ứng dụng. Điều này giúp ẩn đi thông tin máy chủ gốc, tăng tính bảo mật.
- *Cân bằng tải (load balancing):* khi hệ thống có nhiều máy chủ cùng chạy một ứng dụng, Nginx đóng vai trò điều phối viên, phân phối lưu lượng truy cập đều cho các máy chủ để tránh tình trạng một máy bị quá tải, đồng thời tự động bỏ qua các

máy chủ đang gặp sự cố.

- *Bộ nhớ đệm (caching)*: Nginx có thể lưu trữ các bản sao phản hồi từ máy chủ ứng dụng. Khi có người dùng khác yêu cầu cùng một nội dung, Nginx trả về ngay lập tức mà không cần gọi lại ứng dụng phía sau, giúp tăng tốc độ tải trang đáng kể.
- *Chấm dứt kết nối SSL/TLS (SSL termination)*: thay vì để ứng dụng chính xử lý việc mã hóa HTTPS (rất tốn tài nguyên CPU), chúng ta cấu hình Nginx để giải mã SSL ngay tại "cửa ngõ". Sau đó, Nginx giao tiếp với ứng dụng bên trong bằng giao thức HTTP thông thường để tối ưu hiệu năng.

1.3.2 Tại sao cần triển khai SSL/TLS cho máy chủ web

SSL (Secure Sockets Layer) và TLS (Transport Layer Security) là các giao thức mật mã học được sử dụng để bảo mật truyền thông tin trên Internet, giúp mã hóa dữ liệu giữa trình duyệt và máy chủ, ngăn chặn nghe lén hoặc giả mạo. TLS là phiên bản nâng cấp, an toàn và hiện đại hơn của SSL; tất cả các phiên bản SSL cũ (1.0, 2.0, 3.0) đã bị khai tử, và ngày nay, khi nói đến "chứng chỉ SSL", thực chất là đang sử dụng giao thức TLS (đặc biệt là 1.2 và 1.3).



Hình 1.6: Cách thức HTTPS hoạt động

Dưới đây là 4 lý do cốt lõi giải thích tại sao SSL/TLS lại quan trọng đến vậy:

1. Mã hóa dữ liệu (Encryption)

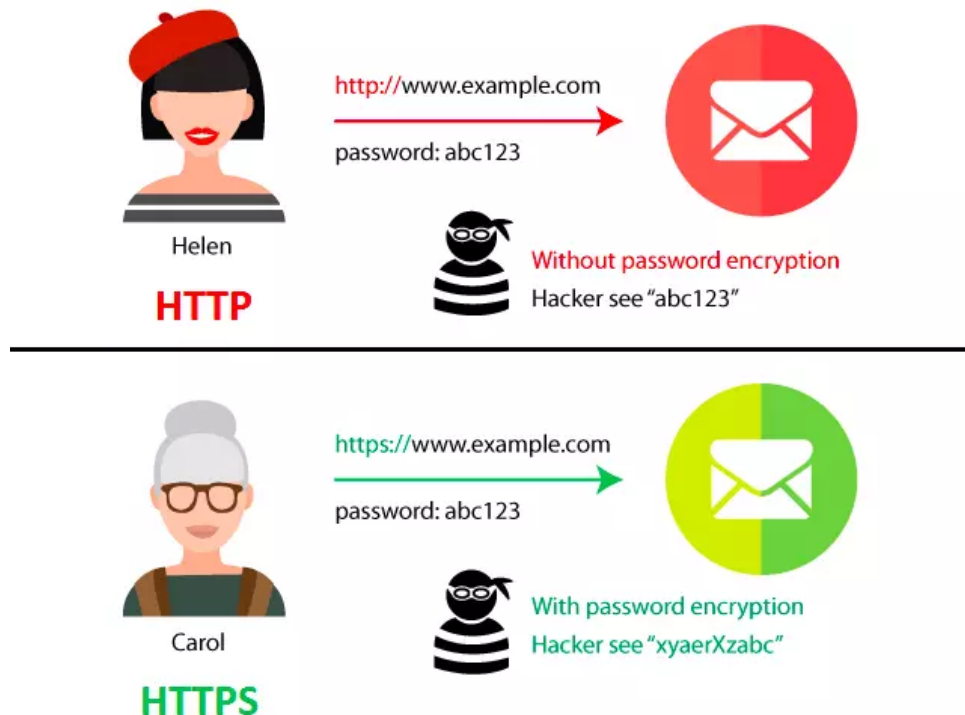
Lý do quan trọng nhất là bảo vệ tính riêng tư. Khi không có SSL/TLS, dữ liệu được truyền đi dưới dạng văn bản thuần túy (Plaintext). Nếu một kẻ tấn công thực hiện kỹ thuật Man-in-the-Middle (người đứng giữa), chúng có thể dễ dàng đọc được mật khẩu, số thẻ tín dụng hoặc thông tin cá nhân của người dùng.

SSL/TLS thực hiện mã hóa dữ liệu, biến thông tin thành những chuỗi ký tự vô nghĩa đối với bất kỳ ai không có khóa giải mã. Ngay cả khi dữ liệu bị đánh cắp trên đường truyền, kẻ tấn công cũng không thể sử dụng được.

2. Xác thực danh tính (Authentication)

Internet là một môi trường đầy rẫy sự giả mạo. Làm sao người dùng biết được họ đang thực sự kết nối với máy chủ của ngân hàng chứ không phải một trang web lừa đảo có giao diện giống hệt?

Chứng chỉ SSL/TLS đóng vai trò như một "chứng minh thư" kỹ thuật số. Nó được cấp bởi các tổ chức uy tín (Certificate Authorities - CA). Khi trình duyệt kết nối với máy chủ qua HTTPS, nó sẽ kiểm tra xem chứng chỉ đó có hợp lệ và thuộc về đúng chủ sở hữu tên miền đó hay không. Điều này giúp ngăn chặn các cuộc tấn công giả mạo trang web.



Hình 1.7: Phân biệt sự khác nhau giữa HTTPS và HTTP

3. Đảm bảo tính toàn vẹn dữ liệu (Data Integrity)

SSL/TLS không chỉ bảo mật mà còn đảm bảo dữ liệu không bị thay đổi trên đường truyền. Nếu một hacker cố tình chen thêm mã độc hoặc thay đổi số tiền trong một giao dịch chuyển khoản khi dữ liệu đang đi qua các trạm trung chuyển, hệ thống SSL/TLS sẽ phát hiện ra sự thay đổi này và ngắt kết nối ngay lập tức. Điều này đảm bảo rằng: Những gì máy chủ gửi đi chính xác là những gì người dùng nhận được.

4. Lợi ích về SEO và niềm tin người dùng

Bên cạnh lý do kỹ thuật, SSL/TLS còn có ý nghĩa về mặt vận hành và kinh doanh:

- **Xếp hạng tìm kiếm (SEO – Search Engine Optimization):** các công cụ tìm kiếm như Google ưu tiên thứ hạng cho các trang web sử dụng HTTPS.
- **Cảnh báo trình duyệt:** các trình duyệt hiện đại (Chrome, Edge) hiển thị cảnh báo "Không an toàn" (Not Secure) đối với các trang web chỉ chạy HTTP, khiến người dùng lo ngại và rời bỏ hệ thống.

Như vậy, khi chúng ta vận hành một hệ thống độc lập, SSL/TLS chính là lớp giáp bảo vệ thành quả lao động của cả tổ chức. Chúng ta có thể cấu hình SSL Termination tại Nginx để tối ưu hiệu suất: Nginx sẽ đảm nhận việc giải mã "nặng nhọc" này, giúp các máy chủ ứng dụng phía sau rảnh tay để xử lý logic nghiệp vụ.

1.3.3 Cài đặt và cấu hình Nginx trên máy chủ Linux

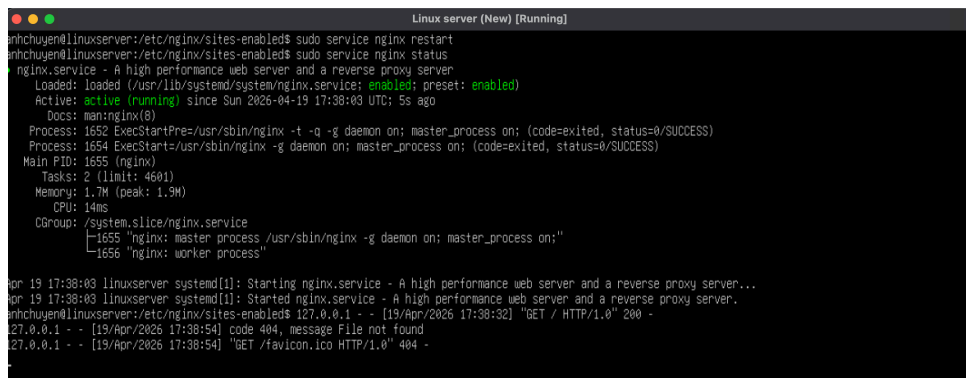
Để cài đặt máy chủ Nginx, có 2 cách: sử dụng gói (package) dựng sẵn hoặc cài đặt từ mã nguồn (source). Cách đầu tiên dễ và nhanh hơn, còn cài đặt từ mã nguồn cho phép bổ sung các module khác để tùy chỉnh Nginx phù hợp với nhu cầu của ứng dụng, đổi lại thao tác phức tạp hơn.

Để cài đặt một gói Debian dựng sẵn, chỉ cần chạy các lệnh:

```
$sudo apt-get update
$sudo apt-get install nginx
```

Sau khi cài đặt xong, có thể kiểm tra bằng lệnh `sudo nginx -v`; lệnh này hiển thị phiên bản Nginx đã cài đặt, ví dụ: `nginx version: nginx/1.24.0`.

Máy chủ web mới sẽ được cài đặt tại `/etc/nginx/`. Nếu vào trong thư mục này, chúng ta sẽ thấy nhiều tệp tin và thư mục. Tệp quan trọng nhất cần chú ý là `nginx.conf` và thư mục `sites-available`. Hãy mở tệp cấu hình của Nginx và nghiên cứu cú pháp, cách thiết lập các tham số trong đó.



```
Linux server (New) [Running]
nhchuyen@linuxserver: /etc/nginx/sites-enabled$ sudo service nginx restart
nhchuyen@linuxserver: /etc/nginx/sites-enabled$ sudo service nginx status
nginx.service - A high performance web server and a reverse proxy server
Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
Active: active (running) since Sun 2026-04-19 17:38:03 UTC; 5s ago
Docs: man:nginx(8)
Process: 1652 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
Process: 1654 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
Main PID: 1655 (nginx)
Tasks: 2 (limit: 4601)
Memory: 1.7M (peak: 1.9M)
CPU: 14ms
CGroup: /system.slice/nginx.service
├─1655 nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
└─1656 nginx: worker process

Apr 19 17:38:03 linuxserver systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Apr 19 17:38:03 linuxserver systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
nhchuyen@linuxserver: /etc/nginx/sites-enabled$ 127.0.0.1 - - [19/Apr/2026 17:38:32] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [19/Apr/2026 17:38:54] code 404, message File not found
127.0.0.1 - - [19/Apr/2026 17:38:54] "GET /favicon.ico HTTP/1.0" 404 -
```

Hình 1.8: Trạng thái hoạt động của máy chủ Nginx

1.3.4 Cấu hình SSL/TLS self-signed trên Linux

Mặc dù chứng chỉ SSL/TLS rất quan trọng đối với website khi đưa lên môi trường Internet, trong môi trường học tập hoặc các hệ thống nội bộ (internal), đôi khi chúng ta không nhất thiết phải mua chứng chỉ từ các tổ chức cấp phép (CA) mà có thể tự mình tạo ra một chứng chỉ "tự ký" (self-signed certificate).

Mặc dù trình duyệt sẽ hiển thị cảnh báo "Kết nối không an toàn" vì nó không biết bạn là ai, nhưng về mặt kỹ thuật, dữ liệu truyền đi vẫn được mã hóa an toàn 100%. Đây là cách tuyệt vời để chúng ta có thể thực hành cấu hình bảo mật trên máy chủ Linux.

Hãy làm theo các bước hướng dẫn trong bài thực hành ở cuối bài.

Bài thực hành

Bài 1. Cấu hình Nginx và thiết lập domain cho website

Mục tiêu lab: Cấu hình thành công Nginx làm Reverse proxy cho website trên máy chủ Ubuntu server; cấu hình và thiết lập HTTP dưới dạng self-sign sử dụng OpenSSL. Các bước thực hiện và câu lệnh:

Chương 1. Tổng quan về triển khai và quản trị hệ thống

Bước 1: Cài đặt các dịch vụ cần thiết Mở terminal hoặc kết nối vào máy chủ Ubuntu qua SSH, sau đó thực hiện các lệnh sau:

```
$sudo apt update && sudo apt upgrade -y
$sudo apt install nginx openssl -y
```

Bước 2: Tạo domain local (truy cập website không cần Internet) Chỉnh sửa file hosts trên hệ thống:

```
$sudo nano /etc/hosts
```

Tiến hành thêm bản ghi sau vào cuối:

```
127.0.0.1    demo.local
```

Sau khi cấu hình, trình duyệt sẽ tự hiểu demo.local chính là máy chủ cục bộ (127.0.0.1).

Bước 3: Chuẩn bị Backend Web Server (cổng 8080) Chúng ta sẽ tạo thư mục và thêm tập tin *.html để làm website giả lập:

```
sudo mkdir -p /var/www/backend
sudo nano /var/www/backend/index.html
```

Nội dung của file index.html là bất kỳ, sau khi lưu nội dung, tiến hành chạy backend (mở một terminal riêng, không đóng). Ta sẽ sử dụng máy chủ web của python để có thể chạy website vừa tạo ở trên. Cú pháp như sau:

```
cd /var/www/backend && python3 -m http.server 8080
```

Sau câu lệnh này, máy chủ web sẽ chạy trên cổng http://localhost:8080 và hiển thị nội dung của tập tin index.html ở trên. *(Hãy tìm hiểu thêm về 2 câu lệnh fg và bg trong Linux để có thể thực thi các lệnh trong chế độ background, như vậy sẽ có thể thực hiện nhiều lệnh đồng thời mà không phải đóng cửa sổ hoặc tắt session của Terminal).*

Bước 4: Cấu hình Nginx Reverse Proxy (HTTP trước) Để cấu hình Nginx, ta cần chỉnh sửa file cấu hình qua trình soạn thảo bất kỳ, có thể là vi, vim hoặc nano:

```
$sudo nano /etc/nginx/sites-available/demo.local
```

Kích hoạt website vừa tạo trở thành mặc định:

```
$sudo ln -s /etc/nginx/sites-available/demo.local /etc/nginx/sites-enabled/
$sudo rm -f /etc/nginx/sites-enabled/default # xóa site mặc định
$sudo nginx -t                               # kiểm tra lỗi cú pháp
$sudo systemctl restart nginx
```

Nội dung file demo.local như sau:

```
server {
    listen 80;
    server_name demo.local;

    # Reverse proxy đến backend
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Như vậy máy chủ Nginx sẽ lắng nghe trên cổng 80, tên máy chủ là demo.local và sử dụng cơ chế proxy để lắng nghe trên cổng 8080 (đây chính là cổng đã sử dụng để chạy máy chủ web python ở trên). Sau khi lưu file cấu hình, ta tiến hành kiểm tra và khởi động lại dịch vụ như sau:

```
$sudo nginx -t           # Kiểm tra lỗi cú pháp
$sudo systemctl restart nginx # Khởi động lại Nginx
```

Kiểm tra kết quả: Mở trình duyệt → truy cập <http://demo.local> Phải thấy trang backend ngay lập tức. Nếu truy cập <https://demo.local> lúc này sẽ bị lỗi (vì chưa cấu hình HTTPS), điều này là bình thường, chúng ta sẽ cấu hình HTTPS trong bài thực hành tiếp theo.

Bài 2. Cấu hình HTTPS cho website sử dụng Openssl

Để cấu hình HTTPS cho website ở bài thực hành 1, ta sẽ cần cấu hình Certificate dạng tự khai báo thông qua Openssl như sau:

Bước 1: Tạo Self-Signed SSL Certificate

Gõ lệnh sau trong terminal để tạo khóa:

```
$sudo mkdir -p /etc/nginx/ssl
$cd /etc/nginx/ssl
$sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout demo.local.key \
-out demo.local.crt \
-subj "/C=VN/ST=ThaiNguyen/O=LabWebServer/CN=demo.local"
```

Giải thích các tham số:

- req -x509: Chỉ định chúng ta muốn tạo một chứng chỉ tự ký theo chuẩn x509.
- nodes: Không mã hóa khóa riêng (để Nginx có thể đọc khóa mà không cần chúng ta nhập mật khẩu mỗi khi khởi động lại).
- days 365: Chứng chỉ có thời hạn 1 năm.
- newkey rsa:2048: Tạo một khóa mới theo thuật toán RSA với độ dài 2048 bit.
- keyout: Nơi lưu trữ Khóa riêng (tuyệt đối không được để lộ file này).
- out: Nơi lưu trữ Chứng chỉ (file này sẽ gửi cho client/trình duyệt).

Nếu các câu lệnh gõ chính xác, khóa và chứng thư số sẽ được tạo ra và lưu trong hai tập tin `demo.local.key` và `demo.local.crt` trong thư mục `/etc/nginx/ssl`

Bước 2: Bật HTTPS trong Nginx

Mở file cấu hình:

```
$sudo nano /etc/nginx/sites-available/demo.local
```

Chỉnh sửa nội dung file cấu hình như sau:

```
server {
    listen 80;
    server_name demo.local;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name demo.local;

    ssl_certificate      /etc/nginx/ssl/demo.local.crt;
    ssl_certificate_key  /etc/nginx/ssl/demo.local.key;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Trong file cấu hình, chúng ta đã khai báo thêm việc server lắng nghe trên cổng 443 và thiết lập đường dẫn tới hai file `.crt` và `.key` để cấu hình SSL.

Thực hiện kiểm tra và khởi động lại dịch vụ nginx:

```
$sudo nginx -t
$sudo systemctl restart nginx
```

Bước 3: Kiểm tra HTTPS (Self-Signed)

Truy cập: `https://demo.local`

- Trình duyệt sẽ hiện cảnh báo “Your connection is not private” (vì cert tự ký).
- Nhấn Advanced → Proceed to demo.local (chấp nhận rủi ro – bình thường trong lab local).

Chúng ta sẽ thấy cùng trang backend → Chứng tỏ reverse proxy + SSL đã hoạt động.

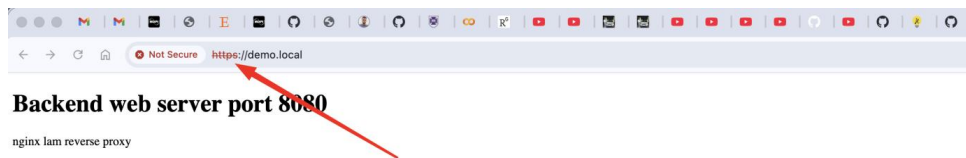
Các lệnh hữu ích để debug khi gặp lỗi:

```
$sudo systemctl status nginx
```

Chương 1. Tổng quan về triển khai và quản trị hệ thống

```
$sudo journalctl -u nginx -xe      # xem log lỗi
$sudo nginx -t                    # kiểm tra config
$curl -I http://demo.local        # test header
```

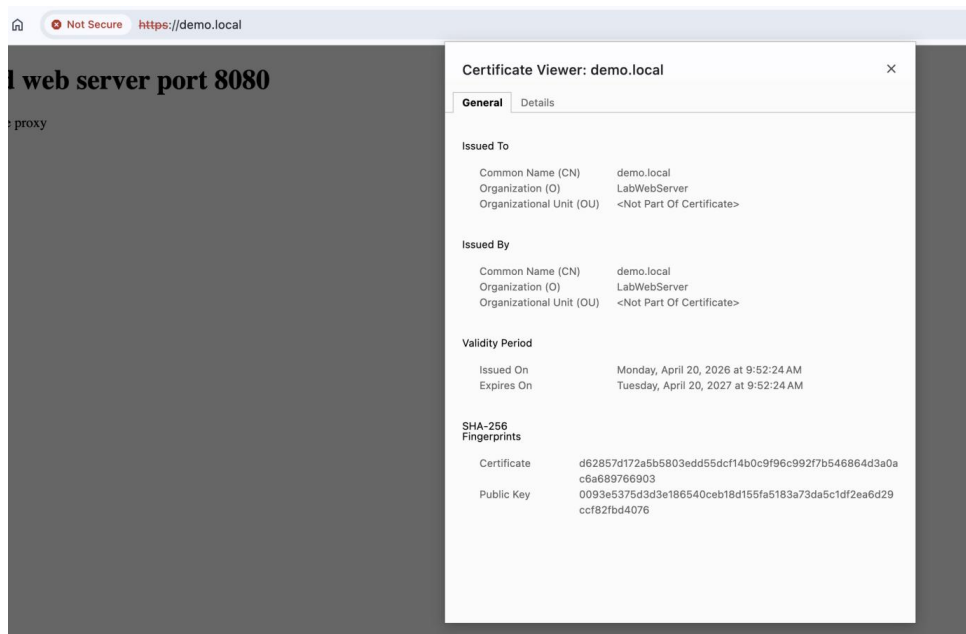
kết quả website hoạt động và hỗ trợ HTTPS:



Hình 1.9: https-webserver

Hình 10: website được cấu hình để hỗ trợ HTTPS

Thông tin chứng chỉ số SSL/TLS:



Hình 1.10: Thông tin chứng chỉ số SSL/TLS

Như vậy chúng ta đã cấu hình thành công website chạy trên tên miền cục bộ và có hỗ trợ HTTPS.

Yêu cầu thực hành và bài tập tự học nâng cao:

- Thay backend Python bằng Apache2/Nginx thứ 2/Node.js/Flask.
- Thêm load balancing (upstream block).
- Thêm basic auth hoặc rate limiting.
- Tạo script tự động start backend (systemd service).

Câu hỏi cuối bài:

Câu 1: Sự đánh đổi giữa Tiện nghi và Bảo mật trong Quản trị Linux Trong bài học về Linux, chúng ta biết rằng tài khoản root có quyền hạn tối cao và dịch vụ SSH cho phép quản trị từ xa. Giả sử em là một quản trị viên hệ thống, em hãy phân tích rủi ro

của việc cho phép đăng nhập trực tiếp bằng tài khoản root qua SSH. Nếu phải thiết kế một quy trình quản trị vừa đảm bảo an toàn (ngăn chặn tấn công Brute-force) vừa đảm bảo hiệu suất công việc cho đội ngũ kỹ thuật, em sẽ đề xuất những biện pháp cụ thể nào?

Câu 2: Triết lý "Gia súc" (Cattle) và "Thú cưng" (Pets) trong tư duy IaC Trong quản trị hệ thống hiện đại, có một khái niệm nổi tiếng: "Coi máy chủ như gia súc (Cattle), không phải thú cưng (Pets)". Dựa trên tư duy Infrastructure as Code (IaC) và nguyên lý Hạ tầng bất biến (Immutable Infrastructure), em hãy giải thích ý nghĩa của sự so sánh này. Tại sao việc "chăm sóc tận tình" (sửa lỗi thủ công, tối ưu hóa riêng lẻ) cho một máy chủ cụ thể lại bị coi là một sai lầm trong vận hành hệ thống lớn? **Câu 3:** Nginx và bài toán tối ưu hóa nguồn lực doanh nghiệp Một doanh nghiệp khởi nghiệp có kinh phí hạn hẹp và sở hữu một máy chủ cấu hình yếu. Họ đang phân vân giữa việc sử dụng Nginx làm Proxy ngược (Reverse Proxy) hay chỉ cài đặt trực tiếp ứng dụng chạy thẳng trên cổng 80. Bằng kiến thức về kiến trúc Event-driven của Nginx và khả năng xử lý tệp tĩnh, em hãy đưa ra lời khuyên cho doanh nghiệp này. Việc thêm một lớp Nginx ở giữa có thực sự làm hệ thống chậm đi do đi vòng, hay nó mang lại lợi ích ngược lại? Hãy chứng minh lập luận của em.

Câu 4: Nghịch lý của Chứng chỉ tự ký (Self-signed Certificate) Chúng ta đã thực hành triển khai SSL tự ký và thấy rằng trình duyệt luôn đưa ra cảnh báo nguy hiểm. Câu hỏi đặt ra là: Trong một mạng nội bộ (Intranet) của một tổ chức, nếu tất cả nhân viên đều được hướng dẫn "nhấn qua" cảnh báo đó để làm việc, điều này sẽ tạo ra một kẽ hở tâm lý như thế nào trong an toàn thông tin? Làm cách nào để một kẻ tấn công có thể lợi dụng thói quen "nhấn qua cảnh báo" này để thực hiện cuộc tấn công Man-in-the-Middle ngay cả khi hệ thống đã có mã hóa? **Câu 5:** Vòng lặp phản hồi trong DevOps và vai trò của Linux Log Trong quy trình DevOps, giai đoạn Giám sát (Monitor) cung cấp dữ liệu cho giai đoạn Lập kế hoạch (Plan). Hãy tưởng tượng em đang đối mặt với một sự cố hệ thống: ứng dụng web chạy trên Nginx thỉnh thoảng bị chậm mà không rõ nguyên nhân. Em sẽ sử dụng những công cụ nào trên Linux để truy vết? Hãy phân tích mối liên hệ giữa việc đọc Log hệ thống, giám sát tài nguyên (CPU/RAM) với việc đưa ra quyết định nâng cấp hạ tầng hoặc tối ưu hóa mã nguồn trong chu kỳ tiếp theo của DevOps.